NetFlow — Complete Deployment Guide

> AI-powered real estate investment intelligence
> Hackathon: The Prompt Engineers Spring 2026

Table of Contents

1. Project Structure
2. Verified Software Versions
3. Prerequisites
4. Environment Variables
5. Quick Start (One Command)
6. Manual Setup (Step-by-Step)
7. Dependency Notes & Known Issues
8. Running in DEMO Mode (No API Keys)
9. API Reference
10. Free API Keys
11. Troubleshooting
12. Architecture

---

1. Project Structure

```
netflow/
├── .env.example                  ← Root env template (copy → .env)
├── .gitignore
├── Makefile                      ← Dev shortcuts (make dev, make install)
├── pyproject.toml                ← Makes `backend` importable as a package
├── start.sh                      ← One-click startup (macOS/Linux)
├── start.bat                     ← One-click startup (Windows)
│
├── backend/                      ← FastAPI application
│   ├── __init__.py
│   ├── .env.template             ← Backend env template
│   ├── config.py                 ← Centralised env config
│   ├── main.py                   ← FastAPI app + all routes
```

```
│   ├── requirements.txt          ← Python deps (pinned)
│   │
│   ├── agents/
│   │   ├── __init__.py
│   │   └── netflow_agent.py      ← LangChain + Ollama scoring agent
│   │
│   └── services/
│       ├── __init__.py
│       ├── fred.py               ← FRED API (30yr mortgage rates)
│       ├── mock_data.py          ← Deterministic mock data (DEMO_MODE)
│       └── rentcast.py           ← RentCast API (listings + rent comps)
│
├── frontend/                     ← Next.js 14 application
│   ├── .env.local                ← Frontend env (NEXT_PUBLIC_API_URL)
│   ├── next.config.js
│   ├── package.json
│   ├── postcss.config.js
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   │
│   └── src/
│       ├── app/
│       │   ├── error.tsx         ← Error boundary
│       │   ├── globals.css       ← Tailwind + Google Fonts
│       │   ├── layout.tsx        ← Root layout
│       │   ├── loading.tsx       ← Loading state
│       │   ├── not-found.tsx     ← 404 page
│       │   └── page.tsx          ← Main page (SSE streaming consumer)
│       │
│       ├── components/
│       │   ├── AIAnalysisCard.tsx
│       │   ├── MarketStats.tsx
│       │   ├── PropertyGrid.tsx
│       │   └── SearchPanel.tsx
│       │
│       └── lib/
│           └── types.ts          ← Shared TypeScript interfaces
│
└── langchain_agent/
    └── netflow_agent.py          ← (legacy copy — canonical is backend/agents/)
```

---

2. Verified Software Versions

All versions below were tested together on macOS 14 (Apple Silicon) and Ubuntu 24 (x86-64). Using different versions — especially for the LangChain stack — may cause silent import errors or dependency conflicts.

System Requirements

Tool	Required Version	Install
Python	3.11 or 3.12	https://www.python.org/downloads/
Node.js	18, 20, or 22 LTS	https://nodejs.org
npm	9+ (bundled with Node)	—
Ollama	0.3.x or 0.4.x	https://ollama.com
Git	any	—

Python Backend — Pinned Versions

Package	Exact Version	Notes
`fastapi`	**0.115.0**	
`uvicorn[standard]`	**0.30.6**	Includes httptools, uvloop, websockets
`starlette`	**0.38.6**	Pinned transitively by fastapi
`sse-starlette`	**2.1.3**	SSE streaming
`httpx`	**0.27.2**	Async HTTP client
`pydantic`	**2.8.2**	
`pydantic-settings`	**2.5.2**	
`python-dotenv`	**1.0.1**	
`langchain`	**0.3.7**	
`langchain-core`	**0.3.15**	
`langchain-ollama`	**0.2.0**	

Package	Exact Version	Notes
`langsmith`	**0.1.147**	⚠️ Must be 0.1.147 exactly — see §7
`tenacity`	**8.5.0**	

Frontend — Pinned Versions

Package	Exact Version
`next`	**14.2.15**
`react`	**18.3.1**
`react-dom`	**18.3.1**
`typescript`	**5.6.3**
`tailwindcss`	**3.4.13**
`postcss`	**8.4.47**
`autoprefixer`	**10.4.20**
`eslint`	**8.57.1**
`eslint-config-next`	**14.2.15**
`@types/node`	**20.16.10**
`@types/react`	**18.3.10**
`@types/react-dom`	**18.3.0**

Ollama Model

Model	Size	Pull command
`llama3`	~4.7 GB	`ollama pull llama3`

3. Prerequisites

macOS / Linux

```bash
# Verify Python (3.11 or 3.12 required — NOT 3.10 or below)
python3 --version

# Verify Node.js (18+ required)
node --version
npm --version

# Install Ollama (skip if already installed)
curl -fsSL https://ollama.com/install.sh | sh

# Pull the Llama 3 model (~4.7 GB one-time download)
ollama pull llama3
```

Windows

```powershell
# Verify Python
python --version

# Verify Node.js
node --version

# Install Ollama from https://ollama.com/download/windows
# Then in PowerShell:
ollama pull llama3
```

Minimum Hardware

	Minimum	Recommended
RAM	8 GB	16 GB
Disk	10 GB free	20 GB free
CPU	4 cores	8 cores / Apple M-series
GPU	Not required	CUDA or Apple Metal (speeds Ollama)

4. Environment Variables

Copy the root template:

```bash
cp .env.example .env
```

Open `.env` and fill in:

```bash
# ── RentCast (free tier: 50 req/month) ──────────────────────
# Get key: https://app.rentcast.io/app/api-access
RENTCAST_API_KEY=your_key_here

# ── FRED — Federal Reserve Economic Data (free, unlimited) ───
# Get key: https://fredaccount.stlouisfed.org/apikey
FRED_API_KEY=your_key_here

# ── LangSmith (optional — traces + evals) ────────────────────
# Get key: https://smith.langchain.com → Settings → API Keys
LANGCHAIN_API_KEY=your_key_here
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=netflow-hackathon

# ── Ollama (change only if running Ollama remotely) ──────────
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3

# ── Demo mode (auto-enabled if both API keys above are blank) ─
DEMO_MODE=false

# ── Frontend ─────────────────────────────────────────────────
NEXT_PUBLIC_API_URL=http://localhost:8000
```

> **Tip:** Leave both `RENTCAST_API_KEY` and `FRED_API_KEY` blank to activate DEMO_MODE automatically — no keys needed to run the app.

5. Quick Start (One Command)

macOS / Linux

```bash
chmod +x start.sh
./start.sh
```

The script automatically:

1. Creates `.env` from `.env.example` if it doesn't exist
2. Starts Ollama and pulls `llama3` if not already present
3. Creates a Python virtualenv and installs all dependencies
4. Starts FastAPI on `:8000`
5. Installs Node modules and starts Next.js on `:3000`

Open **http://localhost:3000** once you see both services ready.

Windows

Double-click `start.bat` or run in PowerShell:

```powershell
.\start.bat
```

Using Make

```bash
make install   # one-time: install all Python + Node deps
make dev       # start Ollama + backend + frontend in parallel
make backend   # backend only
make frontend  # frontend only
make test      # health check both services
make clean     # remove venv, node_modules, .next
```

---

6. Manual Setup (Step-by-Step)

Use this if the quick-start script fails, or you want full control.

Step 1 — Environment config

```bash
cp .env.example .env
# Edit .env — add API keys, or leave blank for DEMO_MODE
```

Step 2 — Python virtual environment

```bash
python3 -m venv venv

# macOS / Linux
source venv/bin/activate

# Windows PowerShell
venv\Scripts\Activate.ps1
```

Step 3 — Install Python dependencies

```bash
# REQUIRED FIRST — makes `import backend.xxx` work
pip install -e .

# Install all pinned dependencies
pip install -r backend/requirements.txt
```

> **Critical:** `pip install -e .` must run before `uvicorn backend.main:app`.
> Without it you get: `ModuleNotFoundError: No module named 'backend'`

Step 4 — Copy env to backend

```bash
cp .env backend/.env
```

Step 5 — Start Ollama (Terminal 1)

```bash
```

```
ollama serve

# First-time only (in a second tab):
ollama pull llama3
```

Skip if `DEMO_MODE=true`.

Step 6 — Start FastAPI backend (Terminal 2)

```bash
# Run from the PROJECT ROOT — not from inside backend/
uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

Verify:

```bash
curl http://localhost:8000/health
# {"status":"ok","service":"NetFlow API","demo_mode":false}
```

Step 7 — Start Next.js frontend (Terminal 3)

```bash
cd frontend
npm install
npm run dev
```

Open **http://localhost:3000**

---

7. Dependency Notes & Known Issues

⚠️ langsmith version — critical constraint

`langsmith 0.2.x` is **NOT compatible** with `langchain 0.3.7`.
pip will resolve the install without errors but `@traceable` fails at runtime.

The only working combination:

```
langchain==0.3.7
langchain-core==0.3.15
langsmith==0.1.147
```

Yanked versions on PyPI (do NOT use):
`0.1.65, 0.1.100, 0.1.130, 0.1.135, 0.1.136, 0.1.140, 0.1.141`

Versions that do not exist on PyPI:
`0.1.148, 0.1.149` — do not reference these anywhere

To install the correct stack from scratch:

```bash
pip install "langchain==0.3.7" "langchain-core==0.3.15" "langchain-ollama==0.2.0" "l
```

⚠️ pyproject.toml build backend

Must use:

```toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
```

Not `setuptools.backends.legacy:build` — removed in setuptools 68+.

⚠️ uvicorn must run from project root

```bash
# ✅ Correct (from project root)
uvicorn backend.main:app --reload

# ❌ Wrong — causes ModuleNotFoundError
cd backend
uvicorn main:app --reload
```

⚠️ Python version minimum

`langchain-ollama==0.2.0` requires **Python ≥ 3.11**.
Python 3.10 and below will fail at install time.

---

8. Running in DEMO Mode (No API Keys)

DEMO_MODE generates realistic, deterministic mock data seeded by ZIP code — the same ZIP always returns the same properties.

Auto-activation: Leave both `RENTCAST_API_KEY` and `FRED_API_KEY` blank in `.env`.

Manual activation:

```bash
DEMO_MODE=true uvicorn backend.main:app --reload
```

In DEMO_MODE:

Setting	Value
Properties per search	20 mock listings
Data	Deterministic, seeded by ZIP
Mortgage rate	Fixed at 7.2%
AI scoring	Rule-based (no Ollama required)
AI summary	Streams with `asyncio.sleep(0.04)` typewriter effect
API calls	Zero (no RentCast, no FRED)

The `/health` endpoint reports `"demo_mode": true` when active.

---

9. API Reference

Base URL: `http://localhost:8000`
Interactive docs: `http://localhost:8000/docs`

`GET /health`

```json
{"status": "ok", "service": "NetFlow API", "demo_mode": false}
```

`POST /api/search`

Synchronous — waits for full results before responding.

Request body:

```json
{
  "zip_code": "75070",
  "budget": 450000,
  "property_type": "SFH",
  "min_beds": 3,
  "strategy": "LTR"
}
```

Field	Type	Options
`zip_code`	string	5-digit US ZIP
`budget`	integer	max purchase price in USD
`property_type`	string	`SFH` | `Multi` | `Condo` | `Townhouse`
`min_beds`	integer	1–4
`strategy`	string	`LTR` | `STR` | `BRRRR` | `Flip`

Response:

```json
```

```
{
  "properties": [
    {
      "rank": 1,
      "address": "123 Main St",
      "zip_code": "75070",
      "price": 425000,
      "est_rent": 2400,
      "cap_rate": 5.8,
      "cash_flow": 312,
      "grm": 14.7,
      "dom": 21,
      "ai_score": 87,
      "beds": 3,
      "baths": 2,
      "sqft": 1850,
      "tags": ["Top Pick", "Cash+ Flow", "Low DOM"]
    }
  ],
  "mortgage_rate": 7.2,
  "market_summary": "Market analysis text...",
  "zip_code": "75070",
  "search_params": {...},
  "demo_mode": true
}
```

`POST /api/search/stream`

Streaming SSE endpoint. The frontend uses this for live updates.

Event types:

Event type	Payload	When
`status`	`{"msg": "Fetching listings..."}`	Progress updates
`properties`	`{"data": [...], "mortgage_rate": 7.2}`	Results ready
`ai_start`	`{}`	AI analysis beginning
`ai_token`	`{"token": "word "}`	Each streaming word
`done`	`{}`	Stream complete
`error`	`{"msg": "..."}`	Error occurred

`GET /api/market/{zip_code}`

```json
{
  "zip_code": "75070",
  "mortgage_rate_30yr": 7.2,
  "median_rent": 2140,
  "avg_days_on_market": 28,
  "vacancy_rate": 4.1
}
```

`GET /api/rate-history?months=12`

```json
{
  "history": [
    {"date": "2025-05-01", "rate": 6.82},
    {"date": "2025-06-01", "rate": 6.94}
  ]
}
```

---

10. Free API Keys

Service	URL	Free Tier	Used For
RentCast	https://app.rentcast.io/app/api-access	50 calls/month	Property listings, rent estimates
FRED	https://fredaccount.stlouisfed.org/apikey	Unlimited	30yr mortgage rate (`MORTGAGE30US`)
LangSmith	https://smith.langchain.com	Developer plan	Agent traces, token usage, evals

All three sign-ups take under 3 minutes each. FRED and LangSmith keys are available immediately. RentCast may take a few minutes to activate.

---

11. Troubleshooting

`ModuleNotFoundError: No module named 'backend'`

```bash
# Fix: run from project root after pip install -e .
pip install -e .
uvicorn backend.main:app --reload
```

`ImportError` involving langsmith / traceable at runtime

Wrong langsmith version. Fix:

```bash
pip uninstall langsmith -y
pip install "langsmith==0.1.147"
```

`Cannot install langchain and langsmith — conflicting dependencies`

Do not try to install `langsmith==0.2.x` with `langchain==0.3.7`. Use `0.1.147`.

`Connection refused` on `localhost:8000`

Backend isn't running. Start it:

```bash
```

```
uvicorn backend.main:app --reload --port 8000
```

`Connection refused` on `localhost:11434`

Ollama isn't running. Either start it or bypass it:

```bash
ollama serve            # start Ollama
# or
DEMO_MODE=true uvicorn backend.main:app --reload
```

Frontend shows `Failed to fetch`

`NEXT_PUBLIC_API_URL` is wrong or unset. Check `frontend/.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

Restart the Next.js dev server after any `.env.local` change.

`npm run dev` fails with ENOSPC (Linux only)

```bash
echo fs.inotify.max_user_watches=524288 | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Port already in use

```bash
# Kill port 8000
lsof -ti:8000 | xargs kill -9

# Kill port 3000
lsof -ti:3000 | xargs kill -9
```

Ollama times out / AI scoring falls back to rule-based

This is expected behaviour on CPU-only machines. The app gracefully falls back to rule-based scoring. To eliminate the timeout entirely, set `DEMO_MODE=true`.

---

12. Architecture

```
Browser / Next.js :3000
        │
        │  POST /api/search/stream  (SSE)
        ▼
FastAPI :8000  (uvicorn + uvloop)
        │
        ├── RentCast API ─────── listings + rent comps
        │         └── mock_data.py  (DEMO_MODE fallback)
        │
        ├── FRED API ──────────── MORTGAGE30US 30yr rate
        │         └── 7.2% fixed  (DEMO_MODE fallback)
        │
        └── NetFlow Agent (LangChain)
                  ├── ChatOllama → llama3 :11434
                  ├── @traceable → LangSmith (optional)
                  └── rule_based_score()  (Ollama fallback)
```

Service URLs Summary

Service	URL
Frontend	http://localhost:3000
Backend API	http://localhost:8000
Swagger / OpenAPI	http://localhost:8000/docs
Ollama	http://localhost:11434
LangSmith	https://smith.langchain.com